EJB

By:
Mr.Satyendra Singh
Department of Computer Science & Engineering,
ABESIT College of Engineering, Ghaziabad.

What is EJB ?

Enterprise Java Beans

- is platform for building ***portable, reusable***, and ***scalable business applications*** using the Java
- is Java components (classes) that executes in a specialized runtime environment called the ***EJB container***

Enterprise Java Bean

- EJB (*Enterprise Java Bean*) is used to develop scalable, robust and secured enterprise applications in java.
- Unlike RMI, middleware services such as security, transaction management etc. are provided by **EJB Container** to all EJB applications.
- The current version of EJB is EJB 3.2. The development of EJB 3 is faster than EJB 2 because of simplicity and annotations such as @EJB, @Stateless, @Stateful, @ModelDriven, @PreDestroy, @PostConstruct etc.
- To run EJB application, you need an *application server* (EJB Container) such as Jboss, Glassfish, Weblogic, Websphere etc.

When use Enterprise Java Bean?

- **Application needs Remote Access**. In other words, it is distributed.
- **Application needs to be scalable**. EJB applications supports load balancing, clustering and fail-over.
- **Application needs encapsulated business logic**. EJB application is separated from presentation and persistent layer.

EJB Container Services

EJB Container provides valuable services for enterprise application development:

- Transactions support
- Security model
- Persistence support
- Remote access
- Web-services
- Interceptors
- Thread Safety and Concurrency control
- Timing services & job scheduling
- Messaging
- Asynchronous interactions

Types of Enterprise Bean Objects

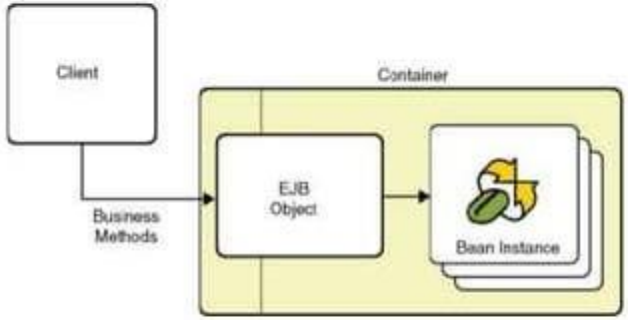
- Session Beans
- Message-driven beans
- Entities

Session Beans

- Performs specific business-logic operations
- Managed by EJB Container
- Available for a client session
- Is removed when the EJB container crashes
- Can be transaction-aware
- Can be invoked either locally or remotely using Java RMI
- Session bean encapsulates business logic only, it can be invoked by local, remote and webservice client.
- It can be used for calculations, database access etc.
- The life cycle of session bean is maintained by the application server (EJB Container).

Session Beans

Session bean object is not directly accessed



Session EJB component has the following elements:

- Component interface
- Implementation class

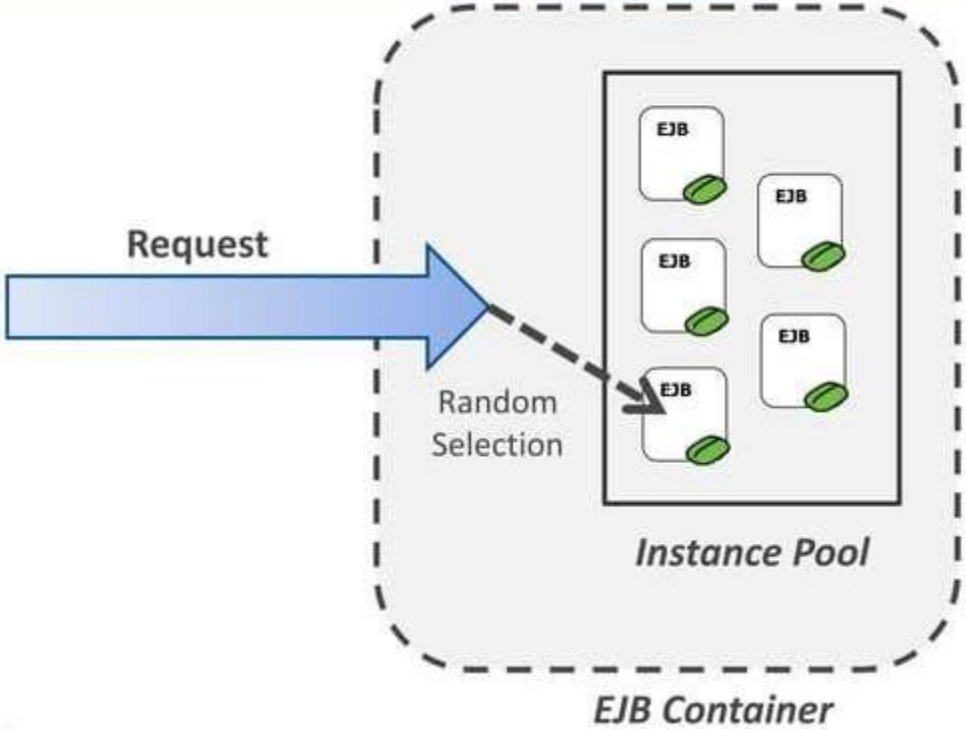
Types of Session Beans

- Stateless Session Beans
- Stateful Session Beans
- Singleton Session Beans

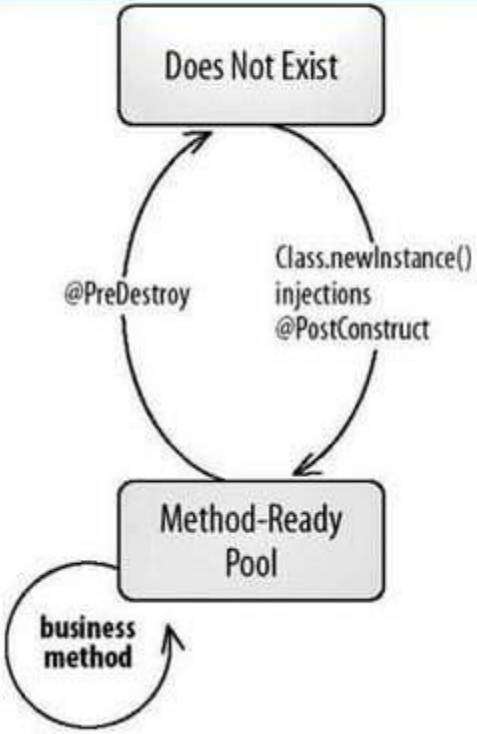
Stateless Session Bean

- Does not retain client-specific information
- A client might not get the same session bean instance
- Can implement a web service
- Any number of requests can be handled by the same instance
  - There are 3 important annotations used in stateless session bean:
  - @Stateless
  - @PostConstruct
  - @PreDestroy

Stateless Session Bean



Stateless Bean Lifecycle and Callbacks




```
@Stateless
public class CalculationBean {

    @PostConstruct
    void construct() {
    }

    @PreDestroy
    void destroy() {
    }

    public int sum(int a, int b) {
        return a + b;
    }
}
```

Stateless Session Bean Example

```
@Local
public interface CalculationLocal {

    int sum(int a, int b);
}


@Stateless
public class CalculationBean implements CalculationLocal {

    public int sum(int a, int b) {
        return a + b;
    }
}
```

Stateful Session Bean

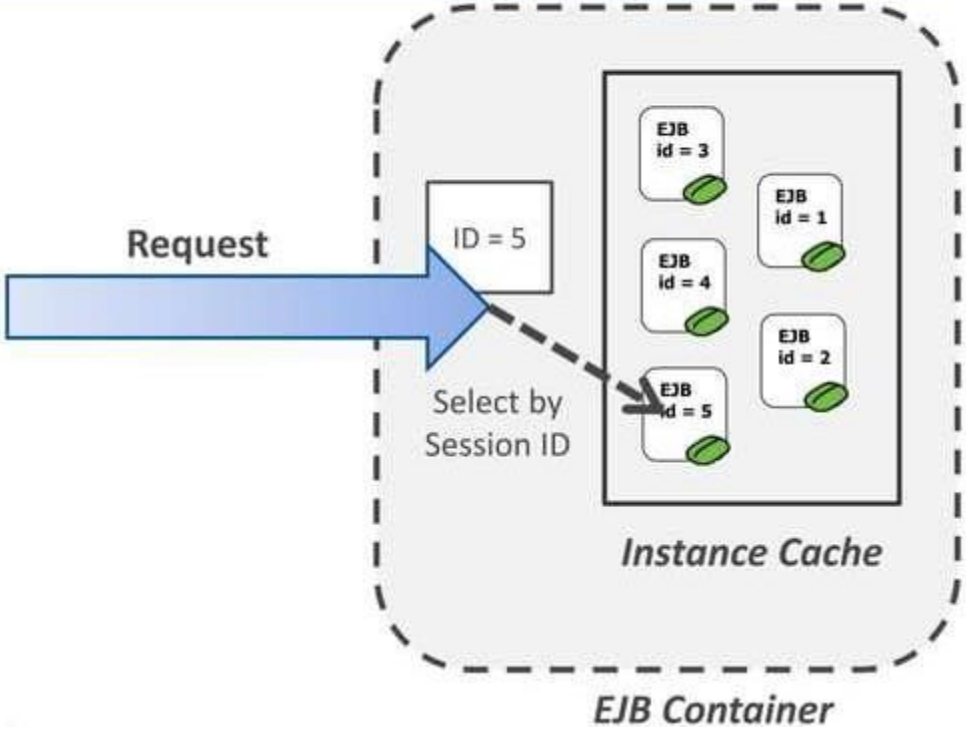
- Supports **conversation state**
- Is never shared, has only one client
- Client can remove bean instance
- Can be passivated by container
- Cannot implement a web service

- **Stateful Session bean** *is a business object that represents business logic* like stateless session bean. But, it maintains state (data).
- In other words, *conversational state* between multiple method calls is maintained by the container in stateful session bean.

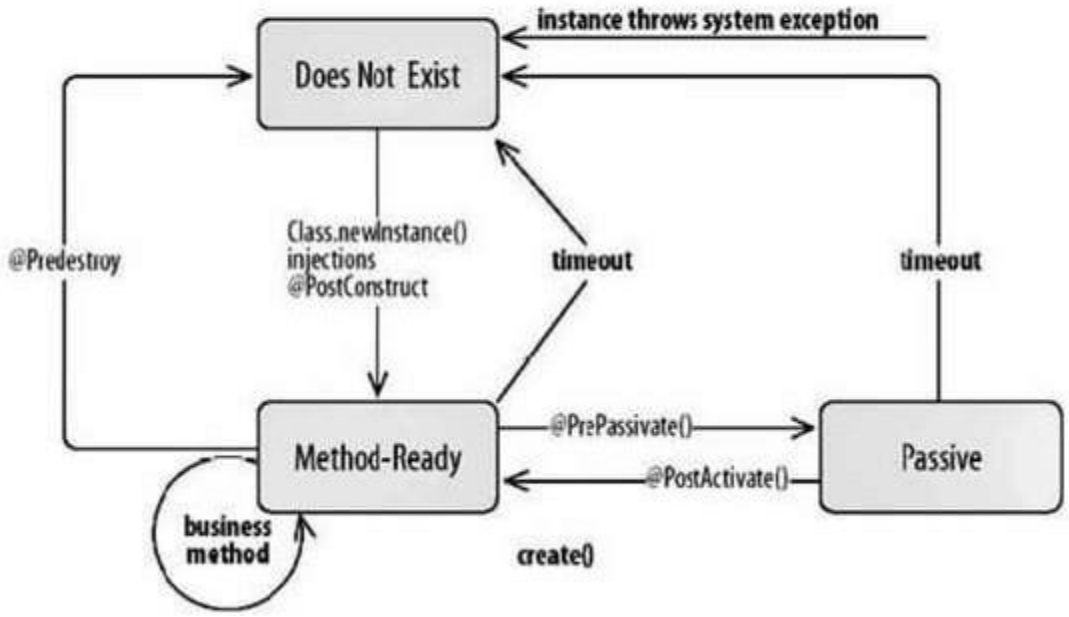
There are 5 important annotations used in stateful session bean:

- @Stateful
- @PostConstruct
- @PreDestroy
- @PrePassivate
- @PostActivate

Stateful Session Bean



Stateful Bean Lifecycle and Callbacks



Stateful Bean Lifecycle and Callbacks (Example)

```
@Stateful
public class CalculationBean implements CalculationRemote {
    private int result = 0;
    @PostConstruct
    void construct(){}
    @PreDestroy
    void destroy(){}
    @PostActivate
    void activate(){}
    @PrePassivate
    void passivate(){}

    @Remove
    public void remove(){}

    public void add(int a) {
        result += a;
    }

    public int getResult() {
        return result;
    }
}
```

Singleton Session Bean

- Instantiated once per application and exists for the lifecycle of the application
- Only one bean per application (jvm)
- Does not maintain state across server crashes/shutdowns
- Supports concurrency requests
- Can implement a web service
- Perform tasks upon application startup and shutdown
- State shared across the application

Initializing Singleton Session Beans

Eager Initialization:

@javax.ejb.Startup

@javax.ejb.DependsOn

```
@Startup
@Singleton
@DependsOn({"PrimaryBean",
"SecondaryBean"})
public class CalculationBean
        implements CalcLocal, CalcRemote {

    private Long result;

    @PostConstruct
    void construct() {
        // any bean initialization ...
        result = 0L;
    }

    .......
}
```

Summary: Session Bean Types

Stateless	Stateful	Singleton
Has no client association	Each client has its own bean instance	Instantiated once per application
Has no state between calls	Stores state between client calls	Each client obtains single state
Pooled in memory	May be passivated to disk, cached	-
Client couldn't manage lifecycle	Removable by client	Client couldn't manage lifecycle
Does not support concurrency	Does not support concurrency	Supports concurrency
Implements WS	Couldn't implement WS	Implements WS

Message-Driven Beans

Message Driven Bean

A message driven bean (MDB) is a bean that contains business logic. But, it is invoked by passing the message. So, it is like JMS Receiver.

MDB asynchronously receives the message and processes it.

A message driven bean receives message from queue or topic, so you must have the knowledge of JMS API.

A message driven bean is like stateless session bean that encapsulates the business logic and doesn't maintain state.

JMS

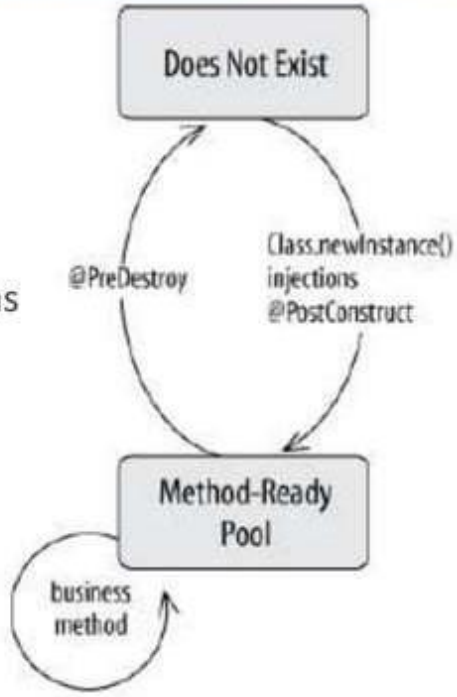
- JMS (Java Message Service) is an API that provides the facility to create, send and read messages. It provides loosely coupled, reliable and asynchronous communication.
- JMS is also known as a messaging service.

Requirement of JMS

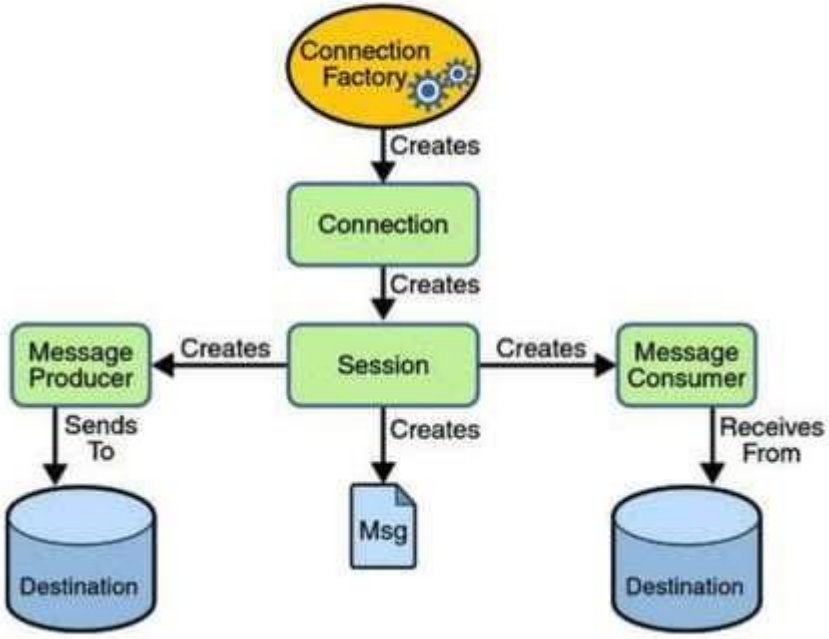
- Generally, user sends message to application. But, if we want to send message from one application to another, we need to use JMS API.
- Consider a scenario, one application A is running in INDIA and another application B is running in USA. To send message from A application to B, we need to use JMS.

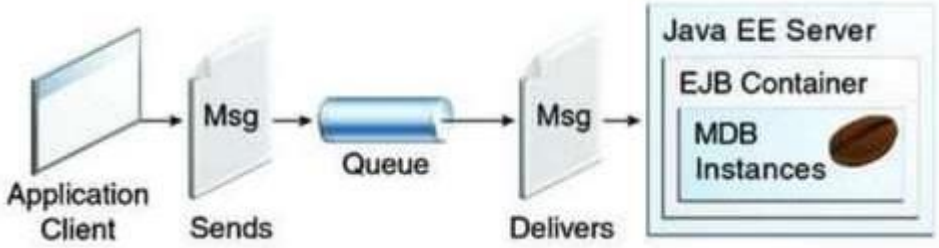
The Lifecycle of a Message-Driven Bean

- The **Method-Ready Pool** is similar to the instance pool used for stateless beans
- When an MDB instance is in the Does Not Exist state, it has not been instantiated yet
- Callbacks : **@PostConstruct, @PreDestroy**



JMS Programming Model




Application
Client
Msg
Sends
Queue
Msg
Delivers
Java EE Server
EJB Container
MDB
Instances

Entity Bean in EJB 3.x

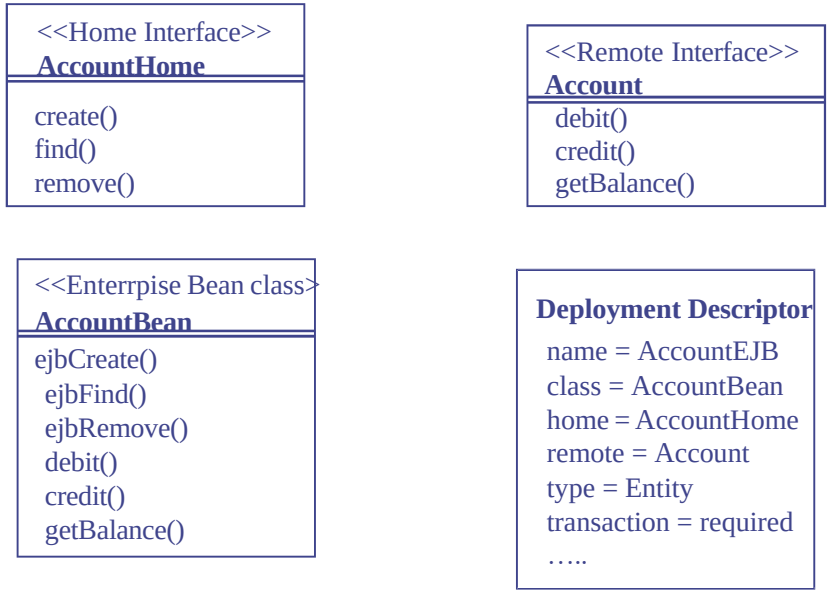
- Entity bean represents the persistent data stored in the database. It is a server-side component.
- In EJB 2.x, there was two types of entity beans: **bean managed persistence** (BMP) and container managed persistence (CMP).

Entity Manager Reference

- The @PersistenceContext annotation can also be placed on a setter method or member field :

```
@Stateless
public class CalculationBean implements CalculationLocal
{
    @PersistenceContext(unitName="MyDB")
    private EntityManager em;
}
```

Enterprise Bean



AccountHome Interface

```
import  java.rmi.RemoteException;
import  javax.ejb.CreateException;
import  javax.ejb.FinderException;
import java.util.Collection;

public interface AccountHome extends javax.ejb.EJBHome {
//create methods
Account create (String lastName, String firstName) throws RemoteException,
CreateException, BadNameException;

Account create (String lastName) throws RemoteException, CreateException;
// find methods
Account findByPrimaryKey (AccountKey primaryKey) throws RemoteException,
FinderException;

Collection findInActive(Date sinceWhen)
      throws RemoteException, FinderException, BadDateException;
}
```

Account Interface

```
import java.rmi.RemoteException;

public interface Account extends javax.ejb.EJBObject
{

BigDecimal getBalance() throws RemoteException;

void credit(BiGDecimal amount) throws RemoteException;

Void debit(BigDecimal amount) throws RemoteException,
                                    InSufficientFundsException;
}
```

AccountBean class

```
public class AccountBean implements javax.ejb.EntityBean {
// life cycle methods from home interface
public AccountKey ejbCreate (String latName, String firstName) throws … {…}
public AccountKey ejbCreate(String lastName) throws …{…}
public AccountKey ejbFindByPrimaryKey(AccountKey primarykey)… {…}
Public Collection ejbFindInactive( Data sinecWhen).. {…}

// business methods from remote interface
public BigDecimal getBalance() {….}
public void credit(BigDecimal amt) {…}
Public void debit(BigDecimal amt) throws InsufficientFundException {….}

// container callbacks from EntityBean container
public ejbRemove( ) throws RemoveException{ …}
public void setEntityContext(EntityContext ec) {…}
public unsetEntityContext(EntityContext ec) {…}
public void ejbActivate() {…}
public void ejbLoad() {….}
public void  ejbStore() {….}
}
```

Deployment Descriptor

```
…
<entity-bean>
   <ejb-name>AccountEJB</ejb-name>
   <home>com.wombat.AccopuntHome</home>
   <remote>com.wombat.Account</remote>
   <ejb-class>com.wombat.AccountBean</ejb-class>
   <persistence-type>Bean<\persistence-type>
   <primary-key-class>com.wombat.AccountKey</primary-key-class>
</entity-bean>

…
<container-transaction>
   <method>
     <ejb-name>AcoountEJB</ejb-name>
     <method-name>*</method-name>
   </method>
   <trans-attribute>required</trans-attribute>
</container-transaction>
```

Questions ?